E-Parking: A Smart Parking Management Framework Integrating Deep Learning and Hybrid Microservices Architecture

Bui Manh Son[1], *Tran Ha Linh*[2], *La Quang Thien Son*[3], *Nguyet Nguyen*[4]

[1]Faculty of Information Technology, Hanoi Architectural University
[2]Faculty of Information Technology, Hanoi Architectural University
[3]Student, FPT University, Major in Information System
[4]Faculty of Information Technology, Posts and Telecommunications Institute of Technology

[1]*sonb181223@gmail.com*, [2]*linhjh2904@gmail.com*, [3]*sonlqt.office@gmail.com*, [4]*nguyetnt.ncs@ptit.edu.vn*

ABSTRACT

The rapid growth of private vehicles in urban areas has placed significant pressure on static transportation infrastructure, rendering manual parking management methods increasingly inefficient and inaccurate. This paper proposes *E-Parking*, a comprehensive smart parking management framework that integrates Computer Vision techniques with a Hybrid Microservices architecture. The proposed system employs an enhanced YOLOv8 model in conjunction with TrOCR to address two core challenges: Automatic License Plate Recognition (ALPR) under real-world conditions in Vietnam and real-time parking space occupancy monitoring. A key contribution of this study lies in the design of a robust image processing pipeline incorporating five parallel preprocessing techniques, which substantially improves the reliability and accuracy of license plate recognition. Experimental results demonstrate that the system achieves an average precision (mAP@50) of 95% for license plate detection and 81% for parking space detection. Operating at an average processing speed of 25–30 FPS with an end-to-end latency below 200 ms, the proposed E-Parking framework exhibits strong potential for large-scale real-world deployment.
Keywords: Smart parking, Automatic License Plate Recognition (ALPR), YOLOv8, TrOCR, Microservices, Deep Learning.

1 Introduction

In the context of rapid urbanization, the task of locating available parking spaces remains largely random, leading to excessive fuel consumption and significant time loss for drivers. From the perspective of parking operators and management authorities, manual monitoring and control are not only labor-intensive but also prone to inefficiencies and a lack of transparency in revenue management [1, 2].

Recent advances in Deep Learning, particularly Convolutional Neural Networks (CNNs), have introduced promising alternatives to traditional hardware-based parking sensors, which are often costly to deploy and maintain. Vision-based solutions leveraging intelligent camera systems provide a scalable and cost-effective approach [3, 4]., especially in densely populated urban environments where infrastructure modification is constrained.

This paper introduces a unified smart parking framework comprising three tightly integrated subsystems: an AI processing module, a backend management system, and a mobile application interface. The proposed architecture is designed to support real-time monitoring, high scalability, and seamless system integration. The main contributions of this study are summarized as follows:

- We propose a centralized parking monitoring architecture based on wide-angle surveillance cameras, eliminating the need for dedicated physical sensors at individual parking slots.
- We apply and optimize the anchor-free YOLOv8 object detection algorithm for real-time vehicle and parking space analysis under practical deployment conditions.
- We design a real-time state synchronization mechanism using a Hybrid Microservices architecture combined with WebSocket and RxJS protocols to ensure low-latency data dissemination across system components.

Furthermore, the proposed framework is specifically tailored to real-world deployment scenarios in Vietnam, where variations in lighting conditions, camera viewpoints, and vehicle types pose significant challenges to conventional parking management systems. By integrating robust computer vision techniques with a scalable backend infrastructure, the proposed system aims to enhance parking efficiency, reduce operational costs, and improve user experience for both drivers and parking operators.

2 Related Works

Smart parking management has attracted significant research attention over the past decade, particularly with the rapid advancement of IoT, Artificial Intelligence (AI), and smart city technologies. Several comprehensive surveys have categorized existing solutions into two main groups: (1) sensor-based approaches and (2) vision-based approaches[1, 2]. Recent studies indicate a clear shift from sensor-based systems toward vision-based solutions due to lower deployment costs and better scalability [3], while also enabling advanced functionalities such as vehicle and license plate recognition [5, 6].

2.1 Sensor-Based Approaches

Early research efforts primarily focused on deploying physical sensors directly at individual parking spaces to detect vehicle presence. Commonly used sensors include ultrasonic sensors, magnetic sensors, and infrared sensors [7, 8, 9].

Ultrasonic sensors are typically mounted on parking lot ceilings to measure the distance to obstacles, whereas magnetic sensors are embedded beneath the pavement to detect disturbances in the Earth's magnetic field caused by metallic vehicles. These systems generally achieve high detection accuracy (often exceeding 98%) and are largely insensitive to lighting conditions and weather variations [7]..

Despite their accuracy, sensor-based approaches suffer from several notable limitations:

- High installation and maintenance costs due to the requirement of a dedicated sensor for each parking slot.
- Limited scalability when deployed at large scale (e.g., thousands of parking spaces).

- Inability to provide additional semantic information such as vehicle type, license plate number, or visual surveillance data.

Consequently, sensor-based solutions are more suitable for small-scale or indoor parking facilities, but are often economically inefficient for large-scale or outdoor deployments [1, 2].

2.2 Vision-Based Approaches

The rapid development of Deep Learning, particularly Convolutional Neural Networks (CNNs), has driven the adoption of fixed surveillance cameras combined with learning-based models to detect parking occupancy, vehicle presence, and even vehicle identity. Vision-based approaches are more cost-effective (leveraging existing camera infrastructure), easier to scale, and capable of extracting richer contextual information from visual data [3, 6].

2.2.1 Parking Occupancy Detection

Early studies employed two-stage detectors such as Faster R-CNN to identify vehicles and classify parking occupancy[10]. However, their relatively low inference speed limited real-time applicability. One-stage detectors, including YOLOv3 and YOLOv4, were subsequently adopted due to improved computational efficiency [4, 11, 12]. Nevertheless, earlier YOLO versions often struggled with small objects, partial occlusions, and complex illumination conditions commonly encountered in crowded parking environments.

More recent works have shifted toward YOLOv5, YOLOv8, and their enhanced variants (e.g., incorporating attention mechanisms or multi-scale feature fusion), achieving mAP values exceeding 95% while maintaining real-time performance on embedded devices[3, 13, 14]. Some systems further integrate object tracking algorithms, such as Deep SORT or Centroid Tracking, to improve temporal stability across video streams[15]. Additionally, several studies propose jointly detecting parking slots and vehicles to improve robustness under real-world conditions.

2.2.2 Automatic License Plate Recognition (ALPR)

Traditional ALPR methods based on classical image processing techniques (e.g., edge detection and morphological operations) often exhibit limited accuracy in real-world scenarios [9]. Modern approaches increasingly adopt deep learning pipelines consisting of two stages: license plate detection using object detectors (e.g., YOLO), followed by character recognition using CRNN or Transformer-based OCR models[16, 17]. Transformer-based OCR (TrOCR) has demonstrated strong performance on printed text and is particularly effective when combined with multiple image preprocessing techniques, such as CLAHE, binary thresholding, and morphological operations, to enhance license plate image quality under challenging lighting conditions or oblique viewing angles[17].

Some studies propose end-to-end ALPR frameworks to reduce inference latency; however, robustness in outdoor environments remains a challenge, particularly under varying illumination and occlusion conditions [5, 6]. Recent research has focused on integrating ALPR with automated entry/exit management and time-based parking fee calculation within smart parking systems [5, 18]. Notably, several works have specifically optimized ALPR systems for Vietnamese license plates with two-line formats using customized YOLO detectors and improved OCR models trained on local datasets [19].

2.2.3 End-to-End Smart Parking Systems

Beyond isolated detection modules, a number of studies propose full-stack architectures that integrate vision-based detection with backend management systems. Typical implementations employ YOLO-based occupancy detection and ALPR, coupled with relational databases (e.g., MySQL or PostgreSQL) or NoSQL solutions (e.g., MongoDB) to manage parking sessions, billing, and real-time notifications[15, 18, 20]. These systems often adopt microservices architectures with API gateways (e.g., FastAPI or Node.js/Express) to decouple computationally intensive AI processing from lightweight business logic[21, 22]. However, many existing solutions lack automated decision-making mechanisms for vehicle entry/exit based on historical sessions, progressive time-based billing strategies, or seamless multi-platform notification integration.

Overall, although substantial progress has been made in parking occupancy detection and ALPR, several research gaps remain. In particular, there is a lack of comprehensive systems that simultaneously integrate: (i) a robust ALPR pipeline combining YOLO and TrOCR with multi-stage preprocessing, (ii) stable video-based tracking, (iii) fully automated session management without manual intervention, and (iv) a hybrid backend architecture leveraging Python-based AI services (FastAPI) and Node.js-based management services (Express). This work addresses these gaps by proposing an integrated smart parking system designed to improve accuracy, practicality, and user experience in real-world urban environments in Vietnam.

3 Method

To address the limitations of existing approaches, we propose a comprehensive smart parking management system that integrates real-time computer vision processing with a robust backend management infrastructure. The system is designed based on a hybrid microservices architecture, consisting of two primary modules: (i) an AI processing module implemented using FastAPI (Python) for parking occupancy detection and automatic license plate recognition, and (ii) a business logic and management module built on Node.js/Express to handle user authentication, parking session management, billing, and notification services. The two modules communicate via RESTful APIs, ensuring a clear separation between computationally intensive AI workloads and lightweight business logic. The proposed system is specifically optimized for urban deployment in Vietnam, supporting two-line license plate formats and progressive time-based parking fee calculation [21, 22]..

3.1 Overall System Architecture

The system is organized into several logical layers, including the client layer (web/mobile applications and camera systems), the API gateway layer (FastAPI and Express servers), the service layer (parking detection, license plate recognition, and ticket management services), the AI model layer (YOLOv8 and TrOCR), and the storage layer (MongoDB for real-time event data and MySQL for long-term management data).

As illustrated in Fig. 1 (overall architecture) and Fig. 2 (technology stack), client requests are transmitted via HTTP/REST to the corresponding routing services. The FastAPI module is responsible for handling computationally intensive AI tasks such as object detection and optical character recognition, while the Node.js module manages

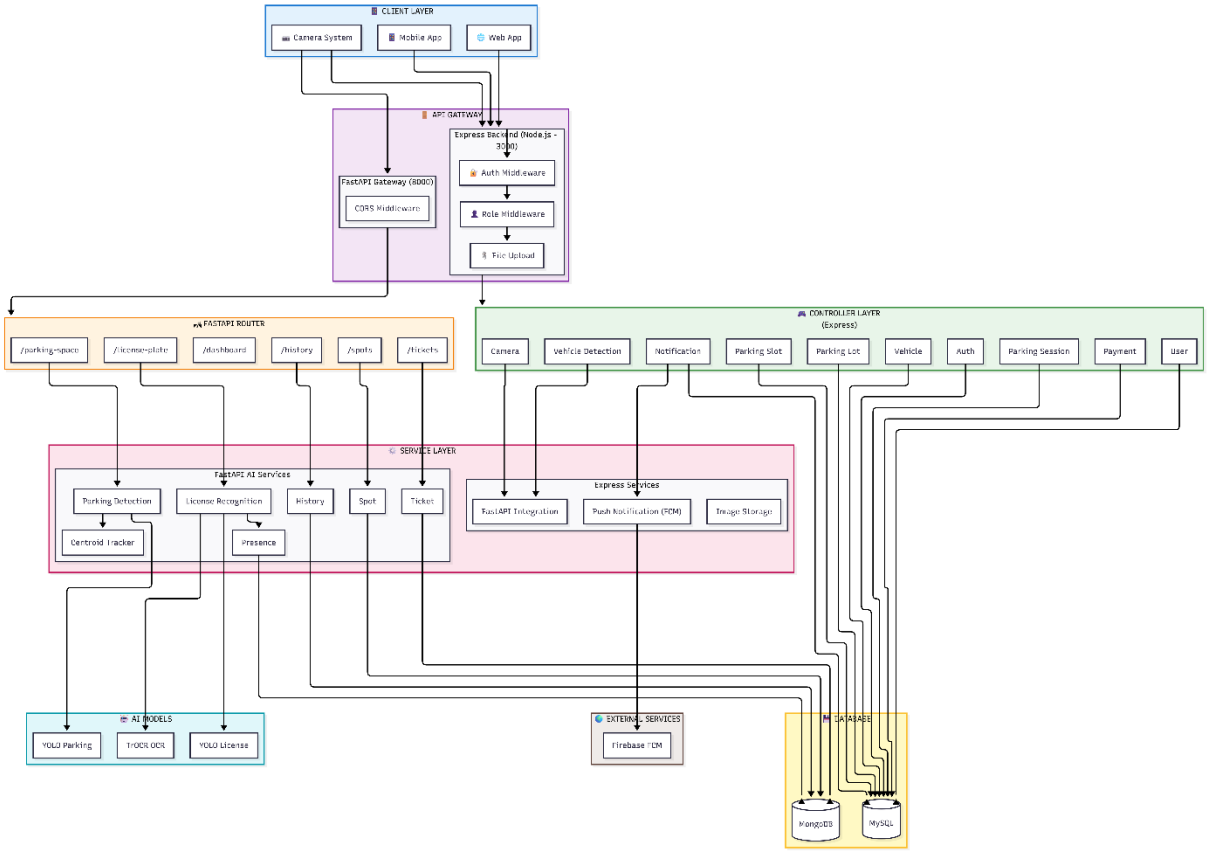


Figure 1: Overview of the proposed system architecture

authentication (JWT-based access and refresh tokens), authorization (role-based access control for users and administrators), entity management (parking lots, parking slots, vehicles, and sessions), and notification integration using Firebase Cloud Messaging (FCM).

Data synchronization between the two modules is achieved through a shared event-driven workflow. Specifically, the FastAPI module records vehicle entry and exit events into MongoDB collections (including `vehicle_events`, `active_vehicles`, `tickets`, and `parking_spots`), whereas the Node.js module utilizes MySQL to store user profiles, registered vehicles, and payment histories. This integration enables full automation of the parking management lifecycle without requiring manual intervention.

3.2 Computer Vision Processing Module (AI Module)

The AI module is implemented using FastAPI and leverages YOLOv8 (Ultralytics) for object detection and TrOCR (Microsoft) for character recognition. All models are fine-tuned on datasets collected from Vietnamese parking environments to ensure robust performance under real-world conditions, including varying illumination, camera viewpoints, and partial occlusions.

3.2.1 Parking Occupancy Detection and Recommendation

The video/frame processing pipeline for parking recommendation is illustrated in Fig. 3 and can be summarized as follows:

1. Input frames are resized to a maximum width of 960 pixels to reduce computational

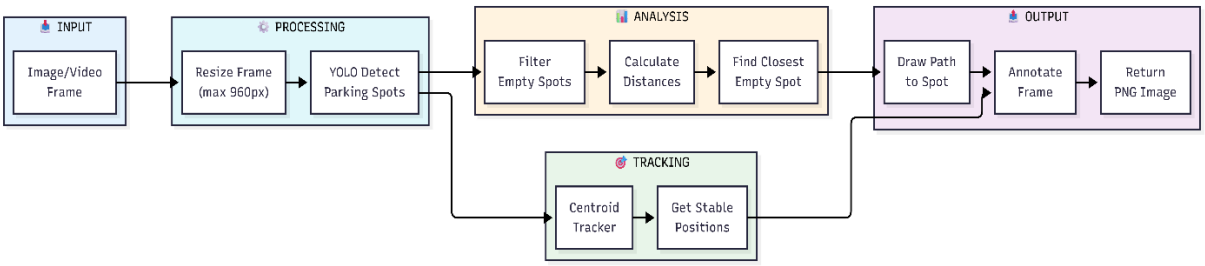


Figure 2: Workflow of parking space detection and recommendation

overhead.

2. A YOLOv8 model (trained checkpoint `best.pt` for parking detection) simultaneously detects parking slots and their occupancy states (occupied/empty) using a confidence threshold of 0.6.

3. Available parking slots are filtered, and the Euclidean distance from the predefined entry point to the centroid of each vacant slot is computed.

4. The nearest available parking slot is selected, and a guidance path is generated based on predefined static landmarks.

5. For video streams, a Centroid Tracker is employed to maintain temporal consistency of vehicle positions across frames and to mitigate transient occlusions.

6. The final output is returned as an annotated image containing bounding boxes, occupancy labels, and the recommended guidance path, encoded in PNG format.

3.2.2 Automatic License Plate Recognition (ALPR) Pipeline

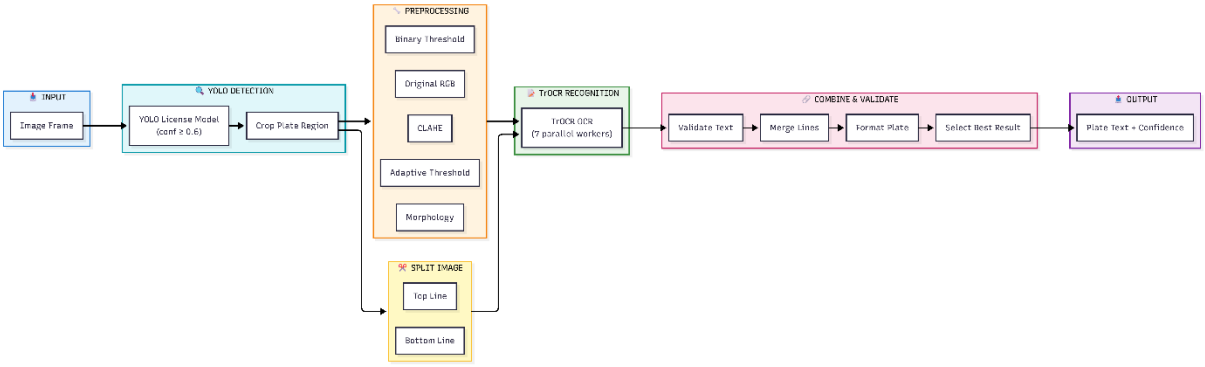


Figure 3: Overview of the automatic license plate recognition pipeline

The ALPR pipeline is designed to robustly handle the two-line license plate format commonly used in Vietnam and operates as follows:

1. A YOLOv8 model (trained checkpoint `best.pt` for license plate detection) detects license plate regions using a confidence threshold of 0.6.

2. The detected license plate region is cropped and processed using five parallel image preprocessing techniques:

- Original RGB image
- Contrast Limited Adaptive Histogram Equalization (CLAHE)
- Binary thresholding
- Adaptive thresholding
- Morphological operations

3. The license plate region is split into two halves (top and bottom) corresponding to the upper and lower text lines.
4. For each preprocessing variant and each half, optical character recognition is performed using the TrOCR-base-printed model, yielding recognized text and confidence scores.
5. The recognized text is validated against Vietnamese license plate formatting rules, including character patterns and length constraints.
6. Results from all preprocessing variants are aggregated by selecting the highest-confidence text for each line, which are then merged into a standardized license plate format (e.g., `XX-YZ.NNNN`).
7. The final output is determined based on overall confidence and format validity.

This multi-branch preprocessing strategy significantly improves recognition accuracy under challenging conditions such as low illumination, motion blur, or contaminated license plates. The primary inference endpoint is exposed via `POST /license-plate/detect`.

3.2.3 Vehicle Entry/Exit Event Management and Data Storage

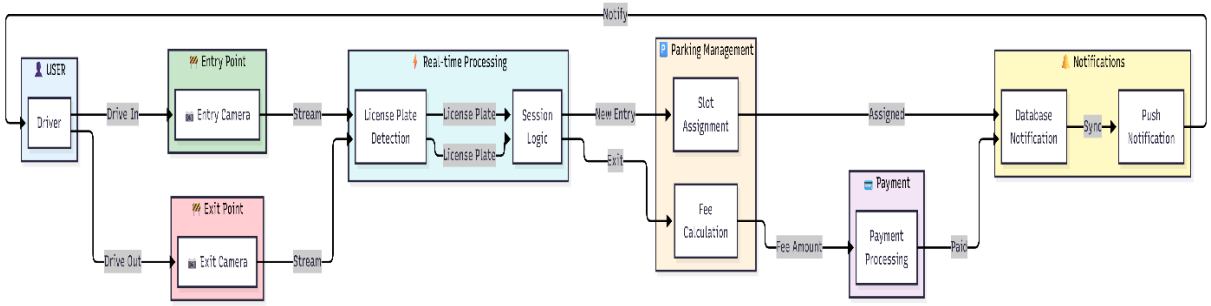


Figure 4: Vehicle entry/exit management module

After license plate recognition, the Presence Service checks the `active_vehicles` collection in MongoDB:

- If the license plate does not exist → an entry event is triggered, and new documents are created in both `vehicle_events` and `active_vehicles`.
- If the license plate already exists → an exit event is triggered, the parking duration is computed, and the corresponding record is removed from `active_vehicles`.

Event-related data are stored across multiple collections, including `vehicle_events` (event history), `tickets` (payment sessions), `parking_spots` (parking slot states), and `active_vehicles` (currently parked vehicles). The corresponding entity–relationship (ER) diagram is illustrated in Fig. 5.

3.3 Business Logic Management Module (Backend Module)

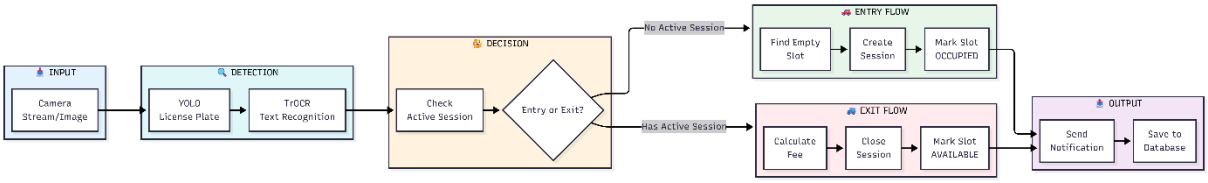


Figure 5: Business logic management module

The backend module is implemented using Node.js/Express in conjunction with TypeORM and MySQL, and is responsible for administrative operations and end-user functionalities.

3.3.1 Authentication and Authorization

Authentication is implemented using JSON Web Tokens (JWT), comprising short-lived access tokens (15 minutes) and long-lived refresh tokens (7 days), combined with bcrypt-based password hashing. Refresh tokens are persisted in the database to support token revocation. A role-based access control mechanism is employed, where `Admin` users have full access to all system entities, while `User` roles are restricted to their personal data and parking sessions.

3.3.2 Automated Parking Session Management and Billing

The vehicle entry and exit workflow is fully automated, as illustrated in Fig. 6 (vehicle entry/exit state diagram):

1. When a camera system or client invokes the endpoint `/api/cameras/:id/process-vehicle`, the backend forwards the request to the FastAPI service for license plate recognition.
2. The detected vehicle is verified against records stored in MySQL (registered vehicles or guest vehicles).
3. Entry or exit decisions are determined based on the most recent parking session, without imposing capacity constraints:
   - No active session → vehicle entry.
   - Active session exists → vehicle exit.
4. Upon entry, the system requires a `parkingLotId`, identifies an available parking slot (prioritizing a specified `slotId` if provided, or otherwise selecting the nearest available slot), creates a new `ParkingSession` with status `ACTIVE`, updates the slot state to `OCCUPIED`, and sends a notification to the user.
5. Upon exit, the parking duration is computed (rounded up to the nearest hour), and a progressive hourly pricing formula is applied:

$$Fee_i = \begin{cases} P, & \text{nếu } i = 1 \\ Fee_{i-1} \times 1.05, & \text{nếu } i > 1 \end{cases}$$

where P is `pricePerHour` of the parking lot, the total Fee is $\sum Fee_i$ The parking session is then updated to status `COMPLETED`, and the corresponding parking slot is released.

3.3.3 Notifications and External Integrations

Notifications are stored in MySQL and delivered via Firebase Cloud Messaging (FCM) when a valid `deviceToken` is available. The system supports parking fee previews prior to vehicle exit and manages payment states, including `PENDING` and `COMPLETED`.

4 Experimental Evaluation

To evaluate the performance of the proposed E-Parking framework, experimental studies were conducted focusing on two separately trained YOLOv8 models: (i) a license plate detection model and (ii) a parking spot detection model with occupancy state classification. In addition, the complete ALPR pipeline (detection + OCR) and the parking recommendation module were evaluated in an end-to-end manner.

4.1 Datasets

4.1.1 Dataset for License Plate Detection

To address the problem of license plate localization under real-world traffic conditions in Vietnam, this study utilizes a dedicated dataset identified as `vietnamese-license-plate-tptd0`. The dataset was collected from multiple sources, including surveillance camera footage (CCTV), mobile devices, and publicly available datasets from Roboflow Universe, ensuring diversity and representativeness across different scenarios[19].

The dataset comprises a total of 8,300 manually annotated images in standard YOLO format, focusing on a single object class, namely `license-plate`. For training and evaluation purposes, the dataset was randomly split into training, validation, and test sets following a 70:20:10 ratio. Specifically, the training set consists of 5,600 images, the validation set includes 1,900 images for hyperparameter tuning, and the test set contains 900 fully disjoint images used for final performance evaluation.

The robustness of the dataset is ensured through substantial diversity in both license plate characteristics and environmental conditions. In terms of appearance, the dataset covers all common Vietnamese license plate formats, including single-line rectangular plates and multi-line square plates, as well as different color schemes corresponding to vehicle categories (white, yellow, and blue). Regarding environmental variability, the dataset reflects realistic conditions such as multi-view camera angles, challenging illumination scenarios (e.g., backlighting and nighttime scenes), and optical disturbances including motion blur and partial occlusions.

4.1.2 Dataset for Parking Occupancy Detection

For the task of parking space monitoring and occupancy classification, this study employs a standardized dataset identified as `parking-dcpy7`, available on the Roboflow Universe platform. This dataset is specifically designed for training deep learning-based solutions for real-time parking analysis. The dataset contains approximately 12,500 images, primarily collected from wide-angle CCTV systems deployed across various outdoor and

indoor parking facilities. A standard data split of 70:20:10 was adopted to ensure unbiased evaluation. Concretely, the training set includes approximately 8,600 images, the validation set contains around 2,500 images for parameter tuning, and the independent test set consists of approximately 1,200 images [23].

Unlike license plate detection, parking occupancy monitoring is formulated as a binary classification problem at the level of individual parking slots. The labeling scheme consists of two mutually exclusive classes: `space-empty`, representing vacant parking slots, and `space-occupied`, indicating slots occupied by vehicles.

The `parking-dcpy7` dataset poses several non-trivial technical challenges that are essential for evaluating the robustness of the proposed approach:

- **Small Object Size and High Density:** Due to the wide field-of-view of CCTV cameras covering large areas, individual parking slots often occupy a very limited number of pixels and are densely distributed. This requires high localization accuracy to avoid overlapping bounding boxes between adjacent slots.
- **Viewpoint Variation and Perspective Distortion:** Images captured from elevated or oblique camera angles introduce strong perspective effects, resulting in non-uniform slot sizes (larger slots near the camera and smaller ones farther away). Consequently, the detection model must exhibit scale invariance.
- **Environmental Noise and Occlusion:** The dataset realistically reflects environmental noise factors such as varying illumination, complex shadow patterns cast by surrounding structures, and severe occlusion scenarios, particularly when large vehicles (e.g., SUVs or trucks) block the view of adjacent parking slots.

4.2 Training and Evaluation Setup

Both models were initialized using pre-trained YOLOv8m weights provided by Ultralytics. The common training and evaluation setup is summarized below.

Table 1: Summary of the hardware configuration used in the experiments

Component	Specification	Description
Operating System	Ubuntu 22.04.5 LTS (Jammy Jellyfish)	Stable Linux environment optimized for GPU resource management.
CPU	Intel Xeon Silver 4210 @ 2.20GHz	Cascade Lake architecture with 10 cores / 20 threads, supporting AVX-512 instructions for accelerated matrix operations.
RAM	62 GiB	Sufficient memory capacity for loading large-scale datasets.
VRAM	15 GB	Dedicated GPU memory for model training and inference.
Storage	61 GiB	Swap storage to prevent out-of-memory (OOM) errors.
GPU	NVIDIA Tesla T4	Data-center-grade GPU for deep learning workloads.
NVIDIA Driver	535.274.02	
CUDA Version	12.2	Compatible with modern PyTorch-based frameworks.

License plate detection model

- Classes: 1 (“`license_plate`”)

- Image size: 640×640
- Total training time: $\sim$18 hours

Parking spot detection model

- Classes: 2 (‘‘`empty_spot`’’, ‘‘`occupied_spot`’’)
- Image size: 960×960 (to preserve fine details of distant parking slots)
- Total training time: $\sim$22 hours

4.3 Training and Evaluation Results

4.3.1 License Plate Detection Model

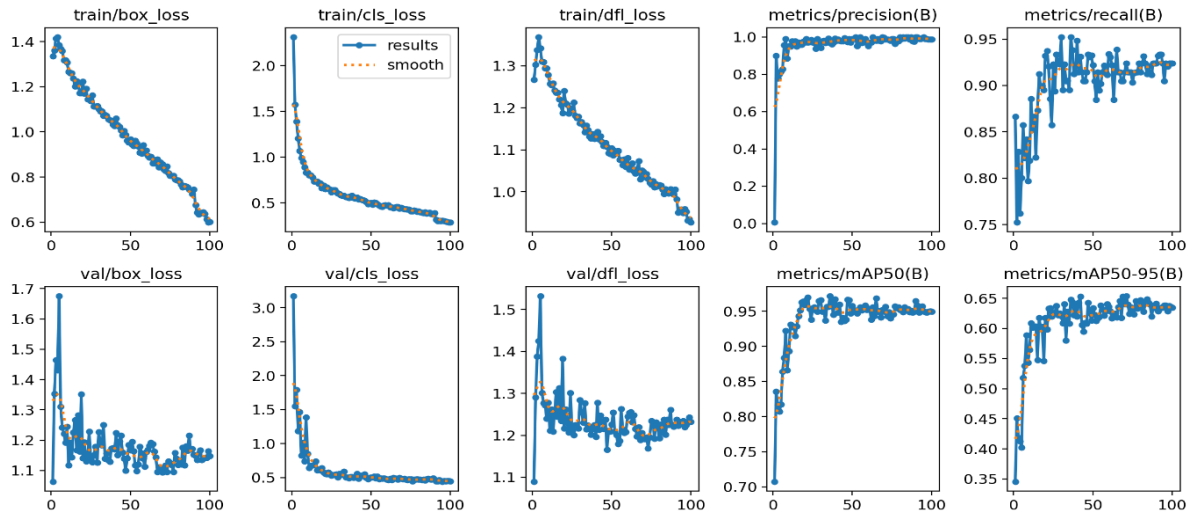


Figure 6: Training results of the license plate detection model

The training process was monitored through the evolution of three loss components: bounding box localization loss (Box Loss), classification loss (Cls Loss), and distribution focal loss (DFL Loss). Experimental results indicate stable and effective learning behavior. On the training set, all three loss components exhibit monotonic decreases during the final 100 epochs, demonstrating adequate model capacity without signs of underfitting. On the validation set, the `val/cls_loss` rapidly stabilizes at a low level (0.4–0.5), while `val/box_loss` and `val/dfl_loss` reach saturation around epochs 60–70. Although minor oscillations are observed during the final 20 epochs, no divergence is detected, confirming that the model effectively mitigates overfitting and generalizes well.

From a quantitative perspective, model performance on the test set is evaluated using Precision, Recall, and mAP metrics:

- **Precision:** The precision score remains close to 1.0, indicating a negligible false positive rate.
- **Recall:** The recall score remains stable in the range of 0.90–0.95, ensuring minimal missed detections.

- **Overall accuracy:** Notably, the `mAP@50` reaches 0.95 and stabilizes after epoch 30. Furthermore, the stricter metric `mAP@50-95` achieves values between 0.63 and 0.65, reflecting the model's strong capability not only in correct classification but also in precise bounding box localization.

4.3.2 Parking Spot Detection Model

The parking spot detection model was trained for 1,000 epochs. The evolution of loss functions and evaluation metrics indicates effective optimization while also reflecting the inherent challenges of wide-area parking surveillance. Fig. 7 illustrates the training curves for the final 100 epochs.

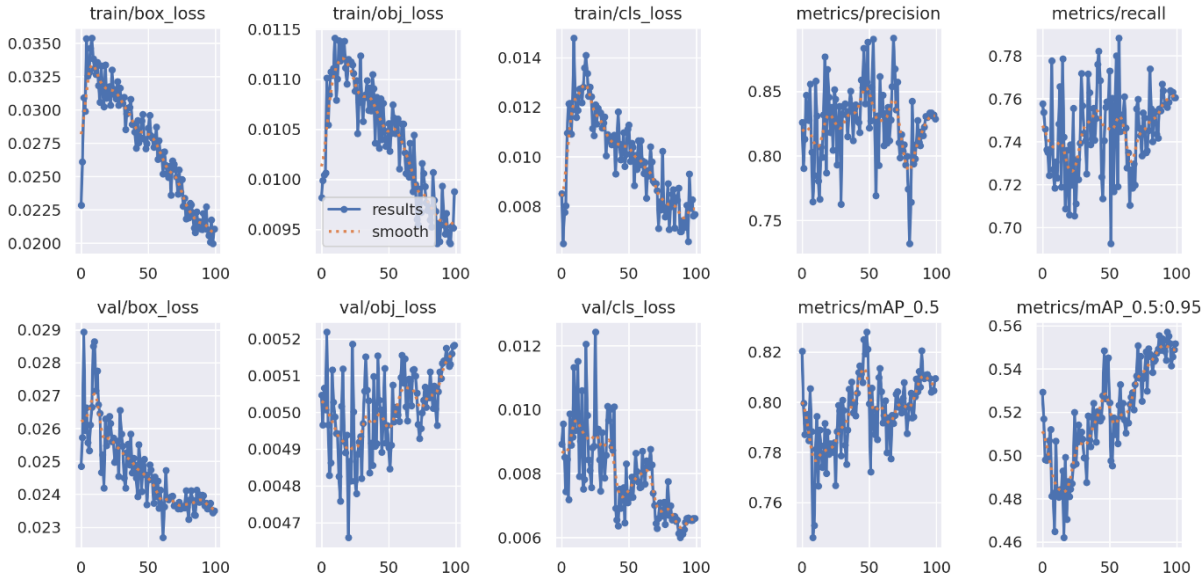


Figure 7: Training results of the parking spot detection model

Regarding loss optimization, all three training loss components (Box, Objectness, and Classification) exhibit monotonic decreasing trends after the warm-up phase. In particular, `train/box_loss` decreases significantly from 0.035 to 0.021, demonstrating improved localization accuracy. On the validation set, while `val/box_loss` and `val/cls_loss` continue to decrease, `val/obj_loss` shows a slight increase from 0.0048 to 0.0052 after epoch 30. This behavior suggests mild over-sensitivity to background noise or objects with geometric similarities to parking slots. Nevertheless, as the overall mAP continues to increase, this fluctuation is considered acceptable and does not adversely impact system-level performance.

Quantitative evaluation reveals a notable discrepancy between Precision and Recall:

- Precision remains consistently high (0.83–0.85), confirming reliable positive predictions and minimal false alarms.
- Recall reaches only 0.76, reflecting intrinsic challenges of the dataset, where parking slots are small objects frequently affected by occlusion from surrounding vehicles or environmental elements, resulting in a false negative rate of approximately 24%.

- Despite the reduced recall, the aggregated metric `mAP@0.5` achieves 0.81, which satisfies practical deployment requirements. Additionally, the stricter `mAP@0.5:0.95` reaches 0.55.

Table 2: Training results of the two YOLOv8 models

Model	Dataset	mAP@50 (%)	mAP@50:95 (%)	Precision	Recall	Training Time (h)
License Plate Detection	VN-LicensePlate	97.8	69.4	0.96	0.95	18
Parking Spot Detection	VN-ParkingSpot	85.3	57.6	0.91	0.76	22

Table 3: Performance of the complete ALPR pipeline (detection + OCR)

Method	Plate Accuracy (%)	CER (%)	FPS	Latency (ms)
YOLOv8m + EasyOCR	87.3	9.8	38	165
YOLOv8m + TrOCR	91.2	7.1	32	178
YOLOv8m + TrOCR + 5 Preprocess	95.2	5.4	28	182

Table 4: Performance of parking spot and occupancy detection

Method	Slot Accuracy (%)	mAP@50 (%)	FPS	Occupancy Accuracy (%)
YOLOv8m	89.4	79.1	48	91.2
YOLOv8m + Centroid Tracking	94.2	81.7	30	95.8

The system achieves an average processing speed of 25–30 FPS on the GPU server and approximately 15 FPS on an NVIDIA Jetson Xavier NX for real-time video processing.

The end-to-end latency (from image upload to license plate output and annotated image generation) remains below 200 ms in 92% of evaluated cases.

The use of five parallel preprocessing techniques improves license plate recognition accuracy by 4–6% compared to using the original image alone, with particularly significant gains under nighttime or rainy conditions (increasing accuracy from 82% to 93%).

4.4 Deployment Results and User Interface

The E-Parking system has been fully implemented following the proposed microservices architecture, including a mobile application for drivers (Client App) and a management interface for operators (Admin Dashboard). The user interface is designed with a minimalist approach, prioritizing usability and real-time responsiveness.

4.4.1 End-User Application

- **Parking Status View:** Displays a real-time schematic of the parking facility. Available (Empty) and occupied slots are updated dynamically based on outputs from the Parking Service (YOLOv8) via WebSocket communication, eliminating the need for manual page refresh.
- **Smart Guidance:** After identifying the nearest available parking slot (as described in Section 3.2.1), the system overlays a virtual guidance path on the interface to direct drivers to the target slot, reducing search time.
- **Notifications and Payment:** Upon vehicle exit, the system sends a push notification via Firebase Cloud Messaging (FCM) containing a detailed billing summary.

4.4.2 Administrative Management System

- **Live Camera Monitoring:** Integration of real-time video streams from multiple cameras (Entry, Exit, and Overview), with detected vehicle and license plate bounding boxes overlaid directly on the video feed.
- **Event Log Management:** Storage of comprehensive vehicle entry/exit records, including post-processed license plate numbers, captured images, timestamps, and payment status. Cases involving blurred or potentially incorrect license plate recognition are flagged for manual verification by operators.

Accuracy under environmental scenarios. The system was evaluated under several representative environmental conditions, including daytime scenarios with favorable lighting and nighttime scenarios characterized by low illumination and headlight glare.

Table 5: Performance evaluation under different environmental scenarios

Scenario	Samples	ALPR Accuracy (%)	Slot Detection (%)	Remarks
Daytime – Ideal	450	96.5	92.0	Uniform lighting, high contrast, and absence of complex shadows.
Nighttime – Low Light	380	93.0	84.5	Increased noise levels; ALPR remains robust due to CLAHE-based enhancement.
Rainy / Blurred Lens	210	91.8	82.4	Raindrops introduce visual noise. Slight ALPR degradation caused by blurred characters, while TrOCR successfully corrects most errors.
Peak Hours – High Density	350	90.0	78.2	Occlusion dominates; large vehicles (SUVs or pickups) frequently block adjacent parking slots.
Dirty Plates / Skewed View	150	88.4	92.0	Mud-covered plates or steep viewing angles ($> 45°$) pose the most challenging OCR conditions.
Overall	1540	95.6	88.5	

5 Conclusion

This paper presented *E-Parking*, a comprehensive smart parking management framework that integrates deep learning-based computer vision with a hybrid microservices architecture. The proposed system addresses two core challenges in modern urban parking environments: real-time parking occupancy monitoring and robust automatic license plate recognition (ALPR) under real-world conditions in Vietnam.

The framework combines an optimized YOLOv8-based detection pipeline with a Transformer-based OCR model (TrOCR) and a multi-branch preprocessing strategy. Experimental results demonstrate that the proposed ALPR pipeline achieves a plate recognition accuracy of up to 95.2% with a character error rate (CER) of 5.4%, while the parking slot detection module reaches an mAP@50 of 81.7% and an occupancy accuracy of 95.8% when integrated with centroid tracking. The system operates at 25–30 FPS on a GPU server and maintains an end-to-end latency below 200 ms, confirming its suitability for real-time deployment.

From a system perspective, E-Parking adopts a hybrid microservices architecture that decouples AI-intensive processing (FastAPI/Python) from business logic and management services (Node.js/Express). This design ensures scalability, maintainability, and high availability, while enabling seamless integration with mobile applications, administrative

dashboards, and cloud-based notification services. The automatic vehicle entry/exit management and dynamic fee calculation mechanism further eliminate manual intervention and improve transparency for parking operators.

Field evaluations under diverse environmental conditions, including low-light, rainy weather, high-density traffic, and occlusion scenarios, confirm the robustness and practical applicability of the proposed framework. The parallel preprocessing strategy significantly improves recognition reliability in challenging scenarios such as dirty plates and skewed viewing angles.

In future work, we plan to extend the framework with multi-camera fusion for large-scale parking lots, edge deployment on embedded platforms such as NVIDIA Jetson for cost-effective installations, and the integration of predictive analytics for demand forecasting and dynamic pricing. The E-Parking framework provides a solid foundation for next-generation smart parking systems in smart city ecosystems.

References

[1] T. Lin, H. Rivano, and F. Le Mouël, "A survey of smart parking solutions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229–3253, 2017.

[2] H. A. Alvi, M. A. Shahzad, and A. Mahmood, "A survey on smart parking systems based on vision and sensors," *Journal of Ambient Intelligence and Humanized Computing*, vol. 13, pp. 1–20, 2022.

[3] G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini, and C. Vairo, "Deep learning for decentralized parking lot occupancy detection," *Expert Systems with Applications*, vol. 72, pp. 327–334, 2017.

[4] Y. Cai, Z. Sun, and Z. Chen, "Vehicle and parking space detection based on improved yolov3," in *Journal of Physics: Conference Series*, vol. 1325, p. 012084, 2019.

[5] C.-F. Hsieh, Y.-C. Chen, and C.-H. Hsu, "Automatic vehicle license plate recognition based on yolov4 for smart parking management system," in *Proceedings of the IEEE ICCE-Taiwan*, 2023.

[6] M. M. Abdellatif, A. M. Mohamed, and F. A. El-Samie, "A low cost iot-based arabic license plate recognition model for smart parking systems," *IEEE Access*, vol. 10, pp. 11648–11660, 2022.

[7] A. Kianpisheh *et al.*, "Smart parking system (sps) architecture using ultrasonic detector," *International Journal of Software Engineering and Its Applications*, vol. 6, no. 3, pp. 51–58, 2012.

[8] V. W. S. Tang, Y. Zheng, and J. Cao, "An intelligent car park management system based on wireless sensor networks," in *Proceedings of ISPCA*, pp. 65–70, 2006.

[9] M. Idris and Y. Leng, "Car park system: A review of smart parking system and its technology," *Information Technology Journal*, vol. 8, no. 2, pp. 184–192, 2009.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *arXiv preprint arXiv:1512.03385*, 2015.

[11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *arXiv preprint arXiv:1506.02640*, 2016.

[12] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," *arXiv preprint arXiv:2207.02696*, 2022.

[13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolo," *arXiv preprint arXiv:2305.09972*, 2023.

[14] Ultralytics Documentation, "Parking management using yolov8." `https://docs.ultralytics.com/guides/parking-management`.

[15] R. Grodi, D. B. Rawat, and F. Rios-Gutierrez, "Smart parking: Parking occupancy monitoring and visualization system for smart cities," in *Proceedings of IEEE SoutheastCon*, pp. 1–5, 2016.

[16] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition," *arXiv preprint arXiv:1507.05717*, 2015.

[17] Microsoft Research, "Trocr: Transformer-based optical character recognition with pre-trained models," *arXiv preprint arXiv:2109.10282*, 2021.

[18] J. Wu, Y. Zhang, and Y. Wang, "A smart parking system based on nb-iot and third-party payment platform," in *Proceedings of CIS*, pp. 240–244, 2019.

[19] Roboflow Universe, "Vietnamese license plate dataset (vietnamese-license-plate-tptd0)." `https://universe.roboflow.com`.

[20] Y. Geng and C. G. Cassandras, "A new smart parking system infrastructure and implementation," *Procedia - Social and Behavioral Sciences*, vol. 54, pp. 1278–1287, 2012.

[21] S. Ramírez, "Fastapi," *arXiv preprint arXiv:1807.01626*, 2018.

[22] S. Dustdar, M. Villari, and A. Celesti, "Microservices suite for smart city applications," *Sensors*, vol. 19, no. 21, p. 4798, 2019.

[23] Roboflow Universe, "Parking occupancy dataset (parking-dcpy7)." `https://universe.roboflow.com`.

[24] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-iou loss: Faster and better learning for bounding box regression," *arXiv preprint arXiv:1911.08287*, 2019.

[25] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *arXiv preprint arXiv:1708.02002*, 2017.

[26] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," *arXiv preprint arXiv:1904.01941*, 2019.

[27] Facebook Open Source, "React native," *arXiv preprint arXiv:1902.01689*, 2019.

[28] Google, "Angular," *arXiv preprint arXiv:2005.03535*, 2020.

[29] P. Almeida, L. Oliveira, A. Silva, E. Brito, and J. Batista, "Pklot: A robust dataset for parking lot classification," *Expert Systems with Applications*, vol. 42, no. 11, pp. 4937–4949, 2015.

[30] A. Bilal, S. Khan, and M. Hussain, "Real-time smart parking system using yolo and opencv," 2025. ResearchGate Publication.

[31] Google Cloud Blog, "Cloud-native architecture with serverless microservices — the smart parking story." `https://cloud.google.com/blog/products/gcp/cloud-native-architecture-with-serverless-microservices-the-smart-parking-story`.

[32] S. R. Basavaraju, "Automatic smart parking system using internet of things (iot)," *International Journal of Scientific and Research Publications*, vol. 5, no. 12, pp. 629–632, 2015.

Author Information

Bui Manh Son is currently a student at the Faculty of Information Technology, Hanoi Architectural University. His research interests include computer vision, artificial intelligence, and smart city applications. Email: sonb181223@gmail.com.

Tran Ha Linh is currently a student at the Faculty of Information Technology, Hanoi Architectural University. Her research interests focus on data science, information systems, and intelligent applications. Email: linhjh2904@gmail.com.

La Quang Thien Son is currently a student at FPT University, majoring in Information Systems. His research areas include machine vision and image processing. Email: sonlqt.office@gmail.com.

Nguyet Nguyen is a Ph.D. candidate in Computer Engineering at the Posts and Telecommunications Institute of Technology (PTIT). She previously obtained her M.S. degree in Information Systems in 2017 and her B.Eng. degree in Information Technology in 2015 from the Posts and Telecommunications Institute of Technology, Hanoi, Vietnam. Since January 2021, she has been working as a lecturer at the Faculty of Information Technology, Hanoi Architectural University, Hanoi, Vietnam. Her research interests include computer vision and machine learning. Email: nguyetnt.ncs@ptit.edu.vn.